

Evaluation

A hands-on course in building agents by building their harness

Lesson 10

Every change is a bet

- Lesson 9's savings: a trim might cut the line that mattered, a summary might lose a fact
- Same for any change: a prompt line, a model, a step limit, a tool
- After the change the agent still runs and the trace still looks normal
- You find out the way you do for untested software: someone uses it, and it's wrong

Evaluation finds out first

Run the agent on tasks whose right outcome you already know, check what happened, and compare with before the change.

1. **Tasks with known outcomes**
2. **Grading**
3. **Comparing**

Grade the world, not the words

- A case is three things: a starting state, a task, a way to tell if the outcome is right
- The outcome is checked **without the agent**: a file, an exit code, the folder afterwards
- An agent that says "done" has proved nothing; a test that passes has
- Any route that ends in the right place passes, two steps or five
- Each case starts from a fresh folder, so cases can't affect each other

Grade by code where you can

- **By code:** a command whose exit status says pass or fail. Cheap, exact, the same verdict every time
- **By a model:** for what code can't read, like an explanation. A second model gets a rubric and the output
- A model grader has its own mistakes, so it's tested too, on answers whose grade you know

A grade you can't trust is worse than none, because you'll believe it.

A count means nothing alone

- The model isn't deterministic: run each case several times and count the passes
- 11 of 12 is good or bad only against something: run the same cases **before and after**
- Record steps, tokens and time too: a change can pass everything and cost twice as much
- Exit non-zero when something got worse, so a git hook or CI job runs it on **every** change

Built on the whole harness, from the outside

- It treats the agent as a box: a task goes in, the world is changed
- That runs all five primitives together, so it catches what only shows when they work together
- **Control flow:** a loop that sets up, runs, grades, records, stops
- The task goes in as **input**; the check runs as **output**; a model grader uses the **model interface**
- Lesson 7's trace says what happened; evaluation says whether it was right

Four cases, each checkable afterwards (abridged)

```
CASES = [  
    {"name": "count", "setup": "seq 1 37 > numbers.txt", "input": "How many lines are in numbers.txt? ...",  
     "check": "[ \"$(tr -d ' \\n' < answer.txt)\" = 37 ]"},  
    {"name": "fix", "setup": "printf ... > calc.py; printf ... > test.py", "input": "test.py fails. ...",  
     "check": "python3 test.py && grep -q 'add(2, 3)' test.py"},  
    {"name": "rename", "setup": "touch a.txt b.txt c.txt", "input": "Rename every .txt file ...",  
     "check": "[ -e a.md ] && [ -e b.md ] && [ -e c.md ] && ! ls *.txt 2>/dev/null"},  
    {"name": "remember", "setup": "true", "input": "Remember that I prefer short answers.",  
     "check": "grep -qi short .quark/memory/memory.md"},  
]
```

Run quark itself, in a fresh folder (abridged)

```
for case in cases:
    where = tempfile.mkdtemp(prefix=f"eval-{case['name']}-")
    subprocess.run(case["setup"], shell=True, cwd=where)
    start = time.time()
    try: subprocess.run([sys.executable, os.path.abspath(__file__), case["input"]], cwd=where, ...)
    except subprocess.TimeoutExpired: pass
    passed = subprocess.run(case["check"], shell=True, cwd=where, ...).returncode == 0
    seconds = round(time.time() - start, 1)
```

- Not a mock: the same file, prompt, guard, sandbox, retries and tracing
- `input="y\n" * 50` answers the guard's questions; its policy still blocks
- Over five minutes, it's killed and graded on what it left behind

The cost comes from the trace (abridged)

```

events = [json.loads(line) for line in open(f"{where}/.quark/traces.jsonl")] if ... else []
models = [e for e in events if e["event"] == "model"]
tokens = sum(e["input_tokens"] + e["output_tokens"] + e["cache_read"] + ... for e in models)
...
    if passed: shutil.rmtree(where, ignore_errors=True)
    else: failed += 1
print(f"{len(cases) - failed}/{len(cases)} passed")
return 1 if failed else 0

if sys.argv[1:2] == ["--eval"]: sys.exit(evaluate(sys.argv[2:]))

```

- Steps are the child run's `model` events; tokens include cache reads
- A failed case keeps its folder; a passed one is deleted
- One new section, caught before `# — input —` reads anything

It remembers last time

```
log, before = ".quark/evals.jsonl", {}  
os.makedirs(".quark", exist_ok=True)  
if os.path.exists(log):  
    for line in open(log): before[json.loads(line)["case"]] = json.loads(line)["passed"]  
cases, failed = [c for c in CASES if not names or c["name"] in names], 0
```

- A case that passed last time and fails now says **REGRESSED**
- Not "it failed", but "it used to pass"
- Run some cases by name: `quark.py --eval fix rename`

A baseline: what normal looks like

```
pass  count      2 steps    7.2s    21342 tokens
pass  fix        3 steps    6.6s    32208 tokens
pass  rename     2 steps    5.8s    21385 tokens
pass  remember   2 steps    4.7s    21422 tokens
4/4 passed
exit 0
```

- 21,000 to 32,000 tokens for tiny tasks: the prompt is about nine thousand, read every step
- `fix` took a step more: it reads the code before changing it

A change that looks harmless

`MAX_STEPS` from 20 to 1, to cap what a run can spend:

```
pass  count      1 steps    3.4s    10654 tokens
FAIL  fix         1 steps    4.6s    10623 tokens  kept /tmp/eval-fix-effj187m  REGRESSED: it passed last time
pass  rename      1 steps    3.5s    10654 tokens
pass  remember    1 steps    4.1s    10700 tokens
3/4 passed
exit 1
```

The failed case kept its folder (shortened)

```
$ cat /tmp/eval-fix-effj187m/calc.py
def add(a, b):
    return a - b
```

Its trace:

```
{"event": "start"}
{"event": "model", "stop_reason": "tool_use"}
{"event": "tool", "cmd": "cat calc.py test.py"}
{"event": "stopped"}
```

The run hit the step limit before the model saw what it read.

What else evaluation can be

- **Cases from reality:** every real mistake becomes a case, found in the traces
- **What passes:** final state, final answer, the path taken, or what must *not* happen
- **Noise and kinds:** pass at least once in k , or every time; regression cases apart from hard ones
- **When:** on every change, on a schedule, and on a sample of live traces
- **As a product:** Braintrust, LangSmith, promptfoo, Inspect run and grade cases, but can't give you the cases

`evaluation.py`: variants, two judges, and trials

Lesson 3's loop as a function, with:

- **Three variants** to compare: `sonnet` (the baseline), `haiku`, and `hasty`
- **Four cases**: three graded by code, `explain` graded by a model against a rubric
- `calibrate()` tests the judge on three known answers before trusting it
- **Jev as judge** beside the LLM as judge: did the agent complete the task?
- **Three trials** of each case, six at a time, and a table against the baseline

Jev as judge (abridged, run shortened)

```
def jev_judge(input, before, after):
    try:
        return jev.system_one({"request": input, "files before": before,
                               "files after": after}, DONE).noul["done"].noul
    except Exception:
        return None
```

```
jev fix          3/3      3/3      3/3
jev log          0/3      0/3      0/3
jev disagrees: sonnet on log, graded 3/3, jev said 0.13, 0.16, 0.15
```

- Sees the folder before and after, never what the agent said; it only reports
- Wrong on `log`: it doesn't count. A disagreement says which runs to read

Cheaper in every column, because it stopped

A baseline copy with `"steps": 1`, called `onestep`:

	sonnet	onestep
count	3/3	3/3
fix	3/3	0/3
log	3/3	0/3
explain	3/3	0/3
passed	12/12	3/12
steps/run	3.2	1.0
tokens/run	2,265	488
seconds/run	5.0	1.3

The grader had a bug

- The baseline passed `explain` only 2 of 3: a sign the **grading** was wrong
- The stored reason showed the judge said `**PASS**`, in bold
- The code checked whether the answer started with `PASS`, so it read a fail
- The judge also hedged: "computes" vs "returns". The rubric was ambiguous
- Fixed both. `calibrate()` hadn't caught it: a grader can pass its check and still fail you

The rule

Write down what a right outcome looks like before you change anything, and then measure it after.

Evaluation runs the primitives, never rebuilds them

- **Control flow:** the loop under test is unchanged; the evaluation's loop is a separate one around it
- **Input:** the agent gets its task the way a person would give it
- **Context:** it doesn't edit the prompt, the memory or the working memory
- **Model interface:** the same calls, to the same models
- **Output:** the tools run as always; the check is one more command beside them

Only the jobs you wrote down

- Four cases all passing is not a good agent; a set goes out of date
- It says something got worse, not why: that's the traces and the episode
- A few trials make noise look like signal
- A model grader can be wrong; a check on it catches some of that
- None of this is solved by a bigger harness: keep reading failures, keep adding cases

Back to the course

Your own agent, its own cases, and the five primitives